

Réponses aux questions des différents TPs

Romain Meniane - Placide Neuilly

TP2

1. Pour le début, on a décidé de créer une particule en 2D pour ce TP. On l'a ensuite vite remplacée par une particule en 3D en utilisant la classe Vecteur créé ultérieurement.
2. Le code est présent en commentaires en dessous du main du programme principal (dans src/main.cpp).
3. On a finalement gardé la deque car après test, jusqu'à 20 000 particules, la deque était celle qui s'exécutait le plus rapidement, après il y avait la list et en dernier le vector.
4. Algorithme de Strömer-Verlet qui est celui présent dans le programme principal (donc src/main.cpp). Il se compile et s'exécute correctement, le fichier txt renvoyé s'appelle « positions.txt ». Il contient à chaque ligne le temps, puis les 12 coordonnées des 4 étoiles. Exemple de la dernière ligne : 468.51 -0.186723 0.00663257 0 0.19 -0.919808 0 -5.06786 -1.57907 0 34.7511 0.00243558 0. Les coordonnées en z sont bien toutes à 0 puisqu'on travaille en 2D dans cette modélisation.
5. Tous les attributs sont en private, de plus on rajoute const aux getters pour s'assurer qu'on ne modifie rien.

TP3

1. L'implémentation de la classe vecteur reprend les notions du tp2 et ne pose aucun problème.
2. Ajout des calculateurs classiques pour un vecteur, ainsi que du += et du produit scalaire pour faciliter et fluidifier le code.
3. Les tests montrent une bonne rapidité de calcul, et ne présente aucune erreur lors de la compilation-execution.
4. Il faut modifier toutes les méthodes qui utilisent les vitesses, les positions ... qui utilisaient avant un Vec2 en 2 dimensions, pour passer à l'utilisation de notre classe Vecteur en 3 dimension. De cette façon, on gagne énormément d'ergonomie sur le code, grâce aux calculateurs, que l'on optimise pour notre problème.
5. Nous avons décider donc de pouvoir faire bouger toutes les particules à chaque instant t, avec la méthode avancer(), qui donc avance naturellement de $v * t$. Pour le calcul des forces, pour l'instant il n'y a que la somme des interactions avec toutes les autres particules de l'univers, car ce sont les seuls forces connus.
6. 7. 8. 9. la création de l'univers ainsi que le test de performance se situe dans le test tp3, on remarque une bonne performance qui s'améliore nettement grâce à l'astuce de $F_{ij} = -F_{ji}$.

10. Une idée plutôt simple à implementer et très intéressante pourrait être de ne prendre en compte que les forces les plus importantes, choisir un certain rayon en fonction de la masse qui définirait ou non la prise en compte de la force d'interaction entre 2 particules.

TP4 Lennard jones

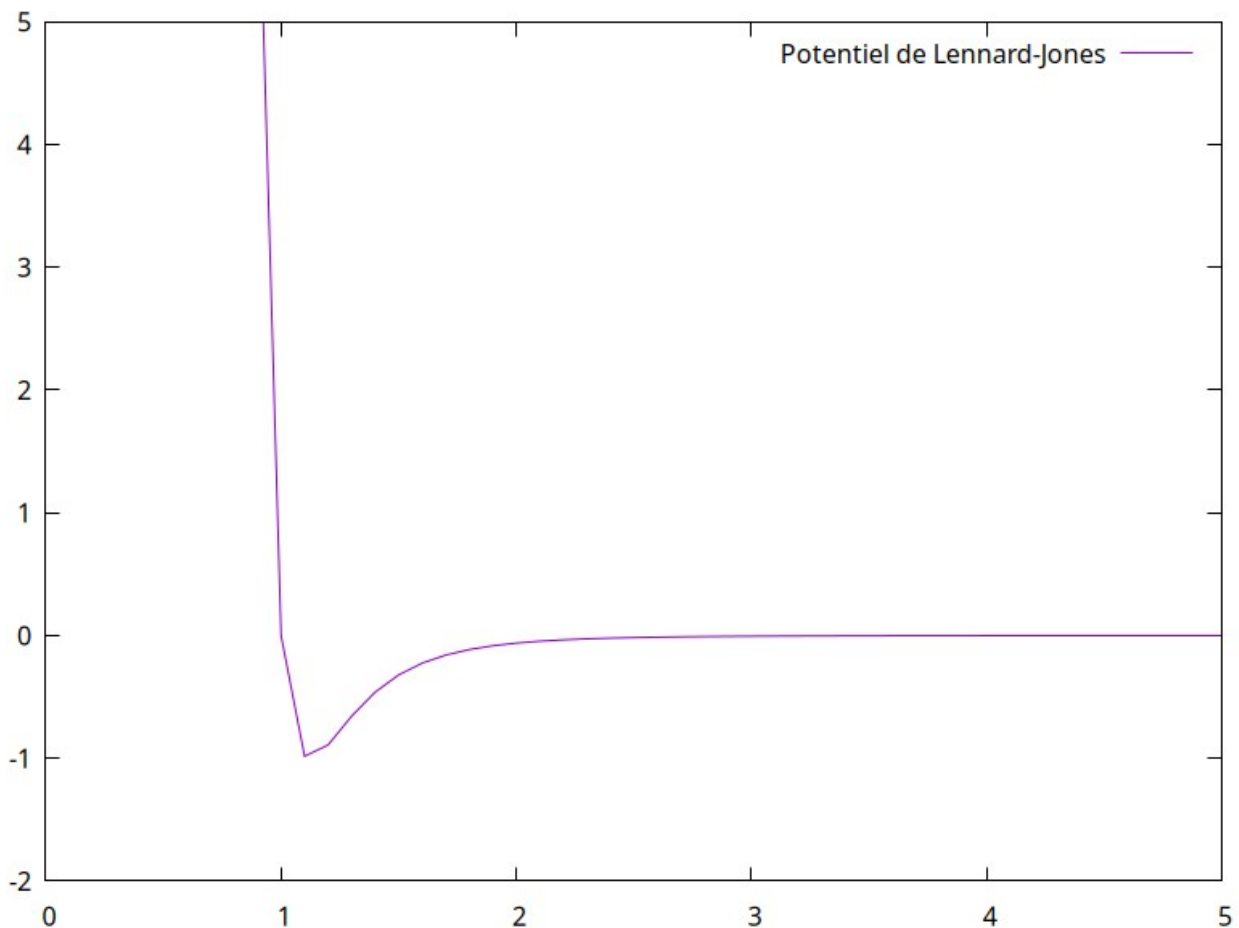
Q1) q2)

Le potentiel entre deux particules séparées d'une distance r est :

$U(r) = 4\epsilon [(\sigma/r)^{12} - (\sigma/r)^6]$ si $r \leq r_{cut}$, 0 sinon.

La force élémentaire correspondante (dérivée du potentiel global, antisymétrique) vaut :

$F_{ij} = 24\epsilon/r^2 \cdot (\sigma/r)^6 \cdot (1 - 2(\sigma/r)^6) \cdot r_{ij}$



Le minimum de potentiel se situe en $r = 2^{1/6} \cdot \sigma \approx 1.1225 \cdot \sigma$, distance utilisée comme espacement initial entre particules à l'équilibre.

Q3) q4)

La classe Univers a été enrichie des champs epsilon, sigma, use_gravity et use_LJ (initialisés par setPhysicsParams). Cela permet de basculer entre simulation gravitationnelle (use_gravity=true, use_LJ=false) et simulation LJ (use_gravity=false, use_LJ=true) sans dupliquer le code.

4. Maillage en cellules et algorithme des voisines

L'espace est découpé en une grille tensorielle de cubes de côté r_{cut} . Le nombre de cellules dans chaque direction d est :

$$ncd_d = \text{floor}(L_d / r_{cut})$$

Chaque cellule stocke des pointeurs vers les particules qu'elle contient et une liste de cellules voisines (jusqu'à 8 en 2D, 26 en 3D).

Algorithme de `calculerForces()` (demi-coquille, Q4) :

Pour chaque cellule C :

Pour chaque paire (i, j) dans C ($i < j$) : calculer F_{ij} .

Pour chaque cellule voisine V de C telle que $\text{adresse}(V) > \text{adresse}(C)$:

Pour chaque particule i de C , j de V :

Si $\|r_{ij}\| \leq r_{cut}$: calculer F_{ij} et appliquer $-F_{ij}$ à j .

La condition $\text{adresse}(V) > \text{adresse}(C)$ évite le double comptage des paires inter-cellules tout en restant correct. Complexité : $O(N \times 27)$ au lieu de $O(N^2)$.

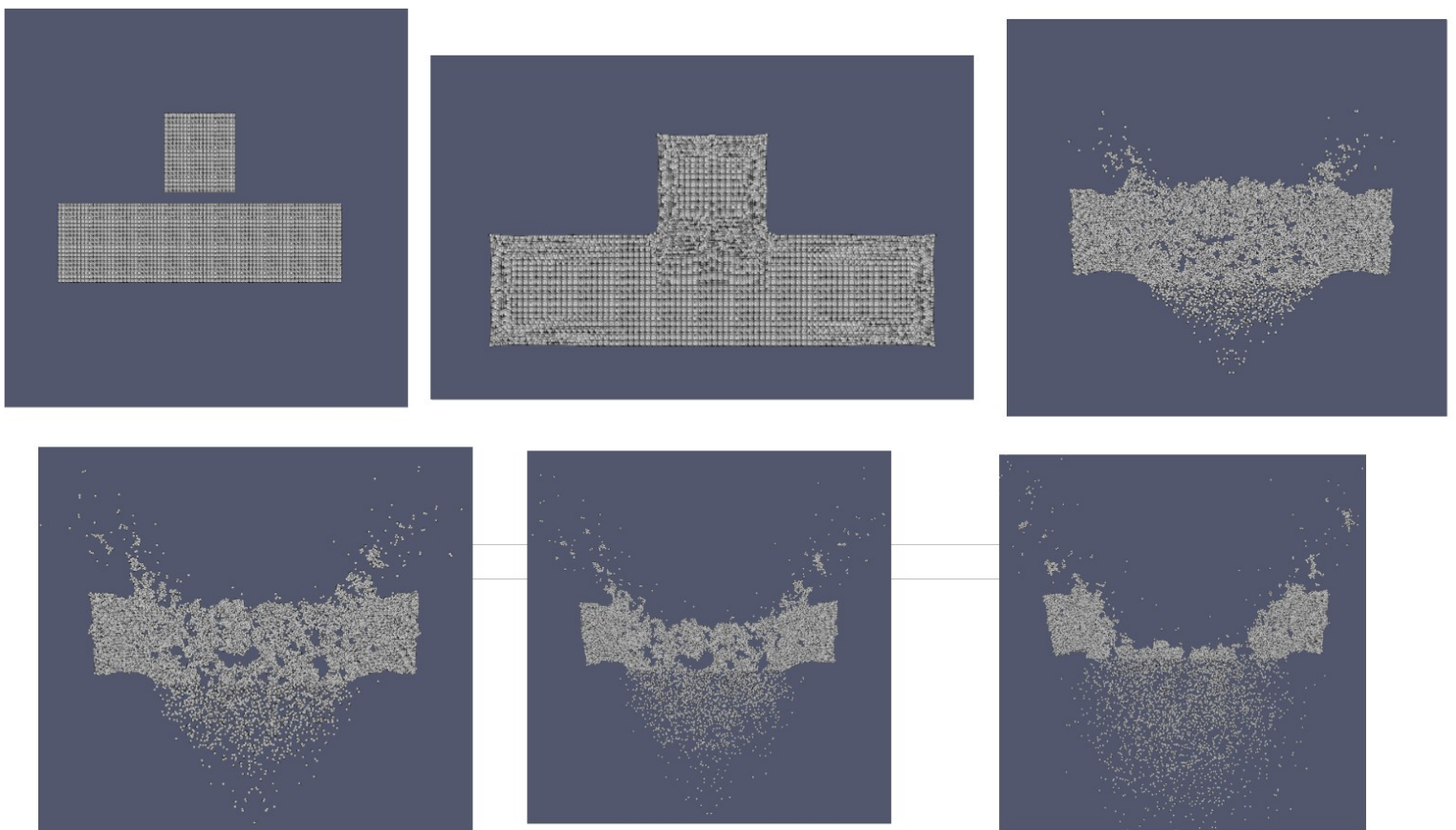
Après chaque pas de temps, `avancer()` met à jour l'appartenance cellulaire de chaque particule qui a changé de cellule (Q5).

q5)

L'algorithme utilisé dans `evoluerVerlet()` est :

1. $x(t+dt) = x(t) + dt \cdot v(t) + dt^2/(2m) \cdot F(t)$ [via `avancer()`]
2. $F(t+dt)$ calculé avec les nouvelles positions [`calculerForces()`]
3. $v(t+dt) = v(t) + dt/(2m) \cdot (F(t) + F(t+dt))$ [`updateVitesse()`]

q6)



Changements après retour du rendu intermédiaire

Il y avait plusieurs modifications à faire après le compte-rendu du rendu intermédiaire. Déjà, tout le code permettant la création de la documentation doxygen. Tout est créé dans le dossier docu/ comme demandé dans les instructions pour le rendu.

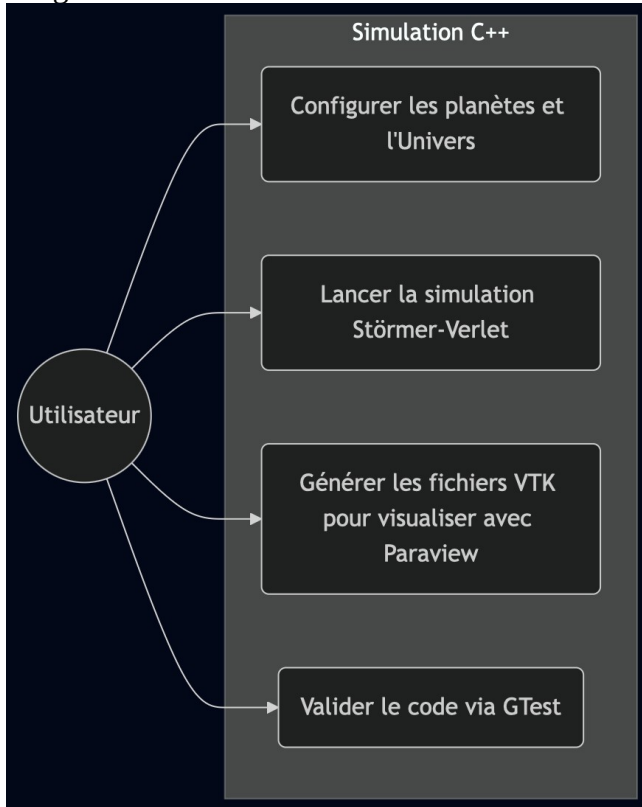
Ensuite, on a du modifié tous nos tests et le main.cpp (qu'on a mis dans le dossier demo/) pour que tout ne soit pas implémenté dans ce fichier. Les fonctions nécessaires (updatePosition, updateVitesse, evoluerVerlet pour l'algo de Stormer Verlet ...) sont désormais dans les classes et plus dans les fichiers de test et de demo.

TP5

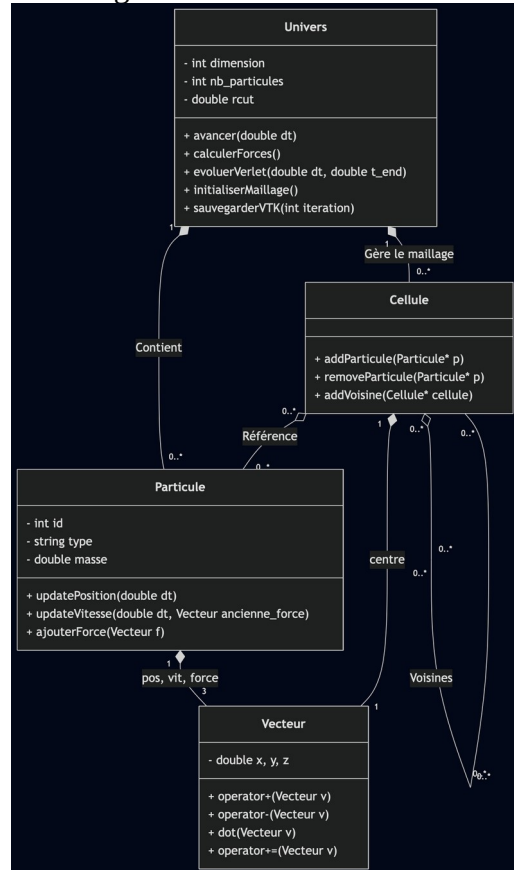
1. 2. Pas de gros problèmes ici, on a d'abord changé le CMakeLists.txt du répertoire racine comme expliqué dans le sujet du tp, puis on a fait pareil pour celui du répertoire tests. On a ensuite créé les tests en commençant par des tests simples sur la classe Vecteur (constructeurs et getters, addition de vecteurs) et la classe Particule (création de particule, ajouterforce...).

3. On a créé une méthode sauvegarderVTK qui va créer un fichier .stu comme indiqué dans le sujet. Ensuite pour tester, on a changé la méthode evoluerVerlet pour qu'elle enregistre les données dans un fichier .stu toutes les 100 itérations (pour qu'il n'y en ait pas trop). Le plus dur était de comprendre comment utiliser Paraview, savoir mettre les bons filtres et les personnaliser pour afficher correctement les planètes.

Diagrammes utilisateur :



Diagrammes Classes :



Diagrammes sequences :

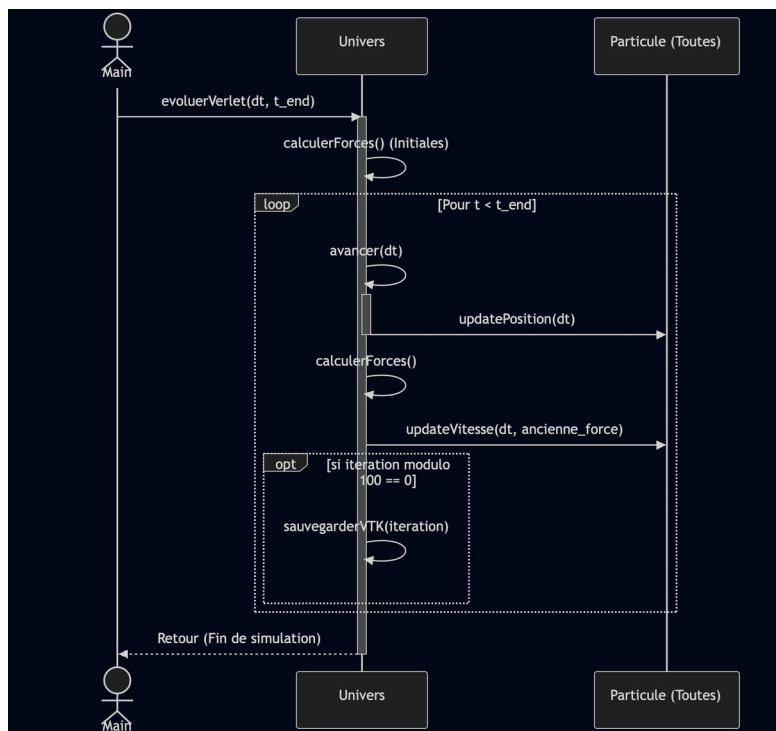
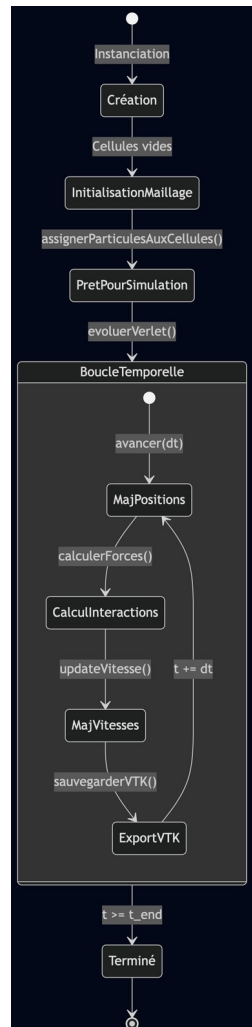
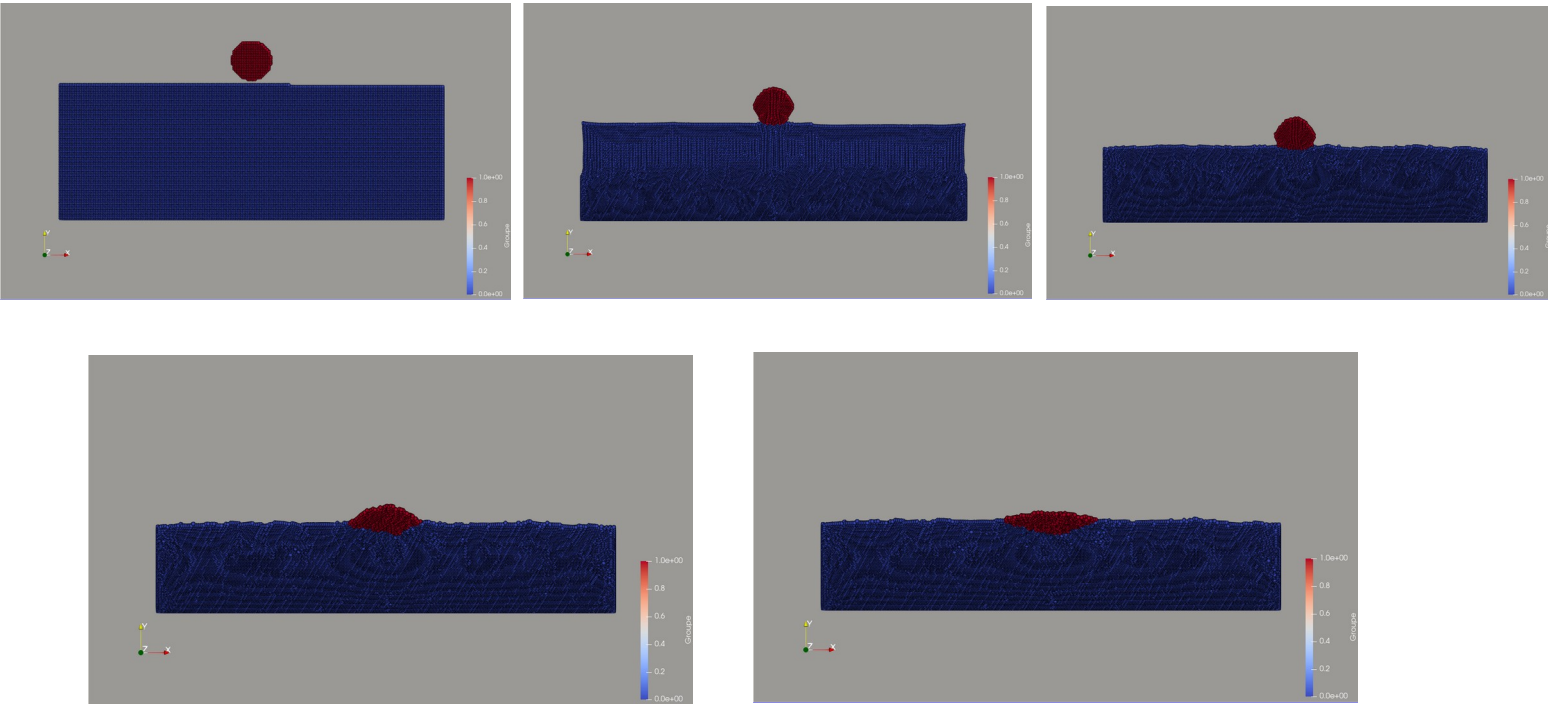


Diagramme transitions :



TP6



1. On a ajouté un enum class `ConditionLimite` dans `Univers.hpp` avec quatre valeurs : `REFLEXION`, `ABSORPTION`, `PERIODIQUE` et `REFLEXION_POTENTIEL`. On configure les conditions par direction via `setConditionsLimites()`. L'application se fait dans `avancer()` en appelant un helper statique `appliquerConditionLimite1D()` qui traite un axe à la fois. Pour la réflexion, on inverse la composante de vitesse perpendiculaire au mur et on repositionne la particule dans le domaine. Pour l'absorption, on marque la particule avec une position sentinelle ($-1e10$) puis on la supprime après la boucle en reconstruisant les assignations cellule->particule pour éviter les pointeurs pendants. Pour le périodique, on ajoute ou soustrait L_d à la position selon le bord touchée.
2. Ajout d'une fonction `demoConditionsLimites()` dans `demo/main.cpp` qui place 25 particules en grille 5×5 avec des vitesses dirigées vers l'extérieur. Au lancement on choisit le type de condition via le terminal. En visualisant les fichiers VTK dans Paraview on vérifie les comportements attendus : rebonds sur les murs pour la réflexion, disparition en touchant le bord pour l'absorption, réapparition du côté opposé pour le périodique.
3. Le potentiel de paroi est implémentée dans `appliquerForcesExternes()` (appelé à la fin de `calculerForces()`). Si la particule est à distance $r < 2^{1/6} \cdot \sigma$ de la paroi, on applique la force du sujet : $F = -24 \cdot \epsilon \cdot (1/2r) \cdot (\sigma/2r)^6 \cdot (1 - 2 \cdot (\sigma/2r)^6)$. La direction dépend du mur : $+y$ pour la parois basse ($y=0$), $-y$ pour la paroi haute ($y=L_y$), $+x/-x$ pour gauche/droite, $+z/-z$ pour avant/arrière.
4. La réflexion simple est instantané et peut créer des artefacts si dt est trop grand : une particule rapide peut traverser le mur en un seul pas de temps. Le potentiel de paroi est plus physique car il crée une zone de répulsion progressive similaire à un vrai mur moléculaire, et s'intègre mieux dans l'algorithme de Verlet puisque la force est continu et dérivable. En pratique pour des dt petits les deux méthodes donnent des résultats proches, mais le potentiel est plus stable numériquement.

5. On a ajouté `use_champ_gravite` et `G_champ` dans `Univers`. La méthode `setChampGravite(double G)` active le champ. Dans `appliquerForcesExternes()`, on ajoute à chaque particule la force $F = (0, m \cdot G, 0)$. C'est distinct de la gravité newtonienne inter-particules (`use_gravity`) qui calcule l'attraction entre chaque paire : ici c'est un champ uniforme externe indépendant des positions relatives des particules.

6. La simulation est dans `simulationCollisionObjets()` dans `demo/main.cpp`. Le sol ($N_2 = 17227$ particules bleu) est positionné en grille rectangulaire en bas du domaine. La balle ($N_1 \sim 395$ particules rouge) est un disque circulaire placé au-dessus avec une vitesse initiale vers le bas. Les parois sont réflexives. Le rescaling des vitesses ($\beta = \sqrt{E_{c_cible}/E_c}$) appliqué toutes les 1000 itérations via `rescalerVitesses()` évite la divergence numérique pendant la simulation jusqu'à $t = 29.5$. Les paramètres sont ceux du sujet : $L_1=250$, $L_2=180$, $\epsilon=1$, $\sigma=1$, $G=-12$, $dt=0.0005$, $E_{c_cible} = 0.005 \cdot (N_1 + N_2)$.

7. On utilise les exceptions standard C++ : `std::invalid_argument` pour les mauvais paramètres (dimension hors $\{1,2,3\}$, $r_{cut} \leq 0$, ϵ ou $\sigma \leq 0$, $dt \leq 0$, $t_{end} \leq 0$) et `std::runtime_error` pour les erreurs d'exécution. Les vérifications sont dans le constructeur, `setPhysicsParams()`, `avancer()` et `evoluerVerlet()`. Le `main()` entoure chaque simulation d'un `try/catch` global.

8. Forces : les exceptions C++ séparent proprement le code nominal de la gestion d'erreur, sans avoir à vérifier des codes de retour à chaque appel. Le type de l'exception (`invalid_argument` vs `runtime_error`) informe sur la nature du problème. Faiblesses : rien n'oblige l'appelant à attraper les exceptions donc une erreur non catchée fait planter le programme sans message explicite. Par ailleurs, des erreurs numériques subtiles (NaN, overflow flottant) ne sont pas détectées par ce mécanisme et se propagent silencieusement dans la boucle de simulation sans lever d'exception, ce qui pourrait nécessiter des vérifications supplémentaires.